OpenGL Outline Algorithms - Detailed Implementation Guide

Project: OpenGLOutline (Win32, OpenGL 3.3 compatibility)

1) Pipeline context

The scene (cube) writes color + depth/stencil into the main render target. Stencil writes value 1 for visible cube fragments. All outline implementations detect outside-edge pixels where center stencil is 0 but nearby stencil has at least one 1. The outline pass writes a composited texture, and the present pass displays it.

2) Brute-force edge condition

The brute-force path uses this binary edge test on outside pixels:

```
outsideEdge = (centerStencil == 0) AND (exists sampled stencil texel > 0). Coverage is
determined by the nearest stencil-marked pixel in the square neighborhood.
```

3) Brute-force mode (square neighborhood)

Shader path: outline.frag (brute-force only). Algorithm: For each pixel, sample all offsets (dx, dy) inside a square radius R around the pixel. If any sampled texel has stencil != 0 and center is 0, color as outline.

Detailed behavior

- Sampling footprint is a full (2R+1) x (2R+1) box. For R=4, that is 81 stencil fetches per pixel.
- The implementation clamps each sample coordinate to framebuffer bounds, so border pixels remain valid.
- Produces thick, robust silhouettes and fills diagonal gaps well.

Complexity

- Per pixel work is O(R^2). Cost grows rapidly as thickness increases.
- Best suited to small-to-medium radii or when quality is preferred over speed.

Visual traits

- Square-neighborhood expansion approximates, but does not match, a Euclidean-radius outline.

4) Jump Flood Algorithm (JFA) mode

Shader paths: jfa_seed.frag, jfa_step.frag, jfa_compose.frag. JFA converts outline detection into nearest-feature propagation, then thresholds Euclidean distance.

Pass A - Seed initialization

- For stencil==1 pixels: store their own pixel center coordinates as seed (x+0.5, y+0.5).
- For stencil==0 pixels: store invalid seed (-1, -1).

Pass B - Jump propagation

- Run multiple fullscreen passes with jump sizes k = 2^n, 2^(n-1), ..., 1.
- In each pass, examine 3x3 neighbors at spacing k, select candidate seed with minimum squared distance.
- Ping-pong between two RGBA32F textures so each pass reads prior state and writes next state.

Pass C - Compose

- For outside pixels, read nearest propagated seed and compute Euclidean distance to current pixel center.
- If distance <= outlineWidth, mark outline; otherwise keep scene color.

Complexity

- Approximate per pixel work is O(log2(max(width,height)) * 9), largely independent of outline thickness.
- Typically scales better than brute-force for thick outlines.

Visual traits

- Distance-based contour is smoother and closer to circular expansion than the square-neighborhood result.

- As an approximation algorithm, can have minor artifacts near sharp corners or very thin structures.

5) Thickness parameter behavior

- UI slider range is 1..100 pixels.
- Brute-force: thickness maps to integer sampling radius.
- JFA: thickness maps to Euclidean distance threshold during compose.
- Because meanings differ, equal numeric values may look slightly different between modes.

6) MSAA x8 interaction

- Scene pass can render into multisample color + depth/stencil textures.
- A resolve pass blits color/depth/stencil into single-sample textures before outline processing.
- This preserves stencil-driven logic while reducing base geometry aliasing.

7) Choosing between these algorithms

- Use Brute-force for consistent thick silhouettes at moderate radii and simpler debugging.
- Use JFA for large thickness values or when smoother distance-like expansion is desired.

8) References

- Khronos OpenGL Wiki - Framebuffer Object https://www.khronos.org/opengl/wiki/Framebuffer_Object
- Khronos OpenGL Wiki - Multisampling https://www.khronos.org/opengl/wiki/Multisampling
- Khronos OpenGL Wiki - Sampler (GLSL) https://www.khronos.org/opengl/wiki/Sampler_(GLSL)
- OpenGL glBlitFramebuffer reference
  https://registry.khronos.org/OpenGL-Refpages/gl4/html/glBlitFramebuffer.xhtml
- Rong & Tan - Jump Flooding in GPU https://www.comp.nus.edu.sg/~tants/jfa.html